

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

Develop Dynamic Webpage Using JavaScript

2. AIM

To develop a dynamic webpage using JavaScript that responds to user actions, by implementing a theme toggle, an image slider, a to-do list, and a live character counter.

3. OBJECTIVE

- To understand how JavaScript is used to make a webpage dynamic and interactive.
 - To use the DOM (Document Object Model) to access and modify HTML elements from JavaScript.
 - To implement a dark/light theme toggle using classList.
 - To build a simple image slider that changes images on button click.
 - To create a to-do list that can add and remove tasks dynamically without reloading the page.
 - To build a live character counter that updates as the user types.
 - To handle user events such as onclick and oninput in JavaScript.
-

4. THEORY

JavaScript is a scripting language that runs in the browser and allows a webpage to respond to user actions after it has loaded, instead of showing only fixed content. This is done through the DOM (Document Object Model), which represents every HTML element as an object that JavaScript can read, change, add, or remove.

Elements are accessed using methods such as `document.getElementById()`, and their content or appearance can be changed using properties like `textContent`, `value`, and `classList`. New elements can also be created and inserted into the page at runtime using `document.createElement()` and `appendChild()`, which is how the to-do list adds new tasks without reloading the page.

User actions are captured using event attributes such as `onclick` (for button clicks) and `oninput` (for typing in a text box). When the event occurs, the linked JavaScript function runs immediately, updating the page — for example, switching the theme, changing the slider image, adding/removing a task, or updating the character count.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file and declare `<!DOCTYPE html>` at the top.
 2. Add a `<style>` block in the `<head>` to style the body, buttons, sections, list items, and textarea.
 3. Add a heading and four `<section>` blocks for the theme switch, image slider, to-do list, and character counter.
 4. In the first section, add a button with an onclick event that calls a `toggleTheme()` function.
 5. In `toggleTheme()`, use `document.body.classList.toggle("dark")` to switch between light and dark mode.
 6. In the second section, add an `<img>` tag and two buttons (Previous, Next).
 7. Store a list of image URLs in an array and keep track of the current index using a variable.
 8. Write `nextImage()` and `prevImage()` functions to update the current index and change the image's src using `document.getElementById()`.
 9. In the third section, add a text input, an Add button, and an empty `<ul>`.
 10. Write an `addTask()` function that reads the input value, creates a new `<li>` with the task text and a Remove button, and appends it to the list.
 11. Add an onclick handler on the Remove button so that clicking it removes that particular `<li>` from the list.
 12. In the fourth section, add a `<textarea>` with an oninput event that calls a `countChars()` function.
 13. In `countChars()`, read the textarea's value length and display it inside a paragraph using `textContent`.
 14. Save the file and open it in a web browser.
 15. Test each feature — theme toggle, image slider, to-do list, and character counter — and record the results as test cases.
-

6. PROGRAM

dynamic.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title> Dynamic Webpage</title>
<style>
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background-color: #f3eded;
  color: #222;
  padding: 100px;
}
body.dark {
  background-color: #0c0b0b;
  color: #f4f4f4;
}
```

```
button {
  padding: 10px 18px;
  margin: 9px;
  font-size: 15px;
  border: none;
  border-radius: 6px;
  background-color: #c48282;
  color: white;
  cursor: pointer;
}
button:hover {
  background-color: #af5a5a;
}
section {
  max-width: 400px;
  margin: 30px auto;
  padding: 20px;
  border: 2px solid #664444;
  border-radius: 20px;
}
body.dark section {
  border-color: #444;
}
#slideImg {
  width: 100%;
  max-width: 300px;
  border-radius: 8px;
}
input[type="text"] {
  padding: 8px;
  width: 60%;
  font-size: 15px;
  border: 1px solid #ccc;
  border-radius: 8px;
}
ul {
  list-style: none;
  padding: 0;
  text-align: left;
}
li {
  background: #f1f1f1;
  color: #222;
  padding: 8px 12px;
  margin: 6px 0;
  border-radius: 5px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
li button {
  background-color: #999;
  padding: 3px 10px;
  font-size: 12px;
}
textarea {
  width: 90%;
  height: 80px;
  padding: 8px;
  font-size: 14px;
}
#charCount {
  font-size: 13px;
  color: gray;
}
</style>
</head>
<body>
<h1> Dynamic Webpage</h1>

<!-- 1. Dark mode toggle -->
<section>
  <h2>Theme Switch</h2>
  <button onclick="toggleTheme()">Toggle Dark / Light Mode</button>
</section>

<!-- 2. Image slider -->
<section>
  <h2>Image Slider</h2>
  
  <br>
  <button onclick="prevImage()">Previous</button>
  <button onclick="nextImage()">Next</button>
</section>

<!-- 3. To-do list -->
<section>
  <h2>Quick To-Do List</h2>
  <input type="text" id="taskInput" placeholder="Add a task...">
  <button onclick="addTask()">Add</button>
  <ul id="taskList"></ul>
</section>

<!-- 4. Live character counter -->
<section>
  <h2>Character Counter</h2>
  <textarea id="textArea" placeholder="Start typing..." oninput="countChars()"></textarea>
  <p id="charCount">0 characters</p>
</section>
```

```
<script>
// 1. Dark mode toggle
function toggleTheme() {
  document.body.classList.toggle("dark");
}

// 2. Image slider
const images = [
  "https://picsum.photos/id/1015/300/200",
  "https://picsum.photos/id/1025/300/200",
  "https://picsum.photos/id/1035/300/200",
  "https://picsum.photos/id/1041/300/200"
];
let currentImage = 0;
function nextImage() {
  currentImage = (currentImage + 1) % images.length;
  document.getElementById("slideImg").src = images[currentImage];
}
function prevImage() {
  currentImage = (currentImage - 1 + images.length) % images.length;
  document.getElementById("slideImg").src = images[currentImage];
}

// 3. To-do list
function addTask() {
  const input = document.getElementById("taskInput");
  const taskText = input.value.trim();
  if (taskText === "") return;
  const li = document.createElement("li");
  const span = document.createElement("span");
  span.textContent = taskText;
  const removeBtn = document.createElement("button");
  removeBtn.textContent = "Remove";
  removeBtn.onclick = function () {
    li.remove();
  };
  li.appendChild(span);
  li.appendChild(removeBtn);
  document.getElementById("taskList").appendChild(li);
  input.value = "";
}

// 4. Character counter
function countChars() {
  const text = document.getElementById("textArea").value;
  document.getElementById("charCount").textContent = text.length + " characters";
}
</script>
</body>
</html>
```

7. CODE EXPLANATION

Code / Function	Purpose
<code>classList.toggle("dark")</code>	Adds the "dark" class to the body if it is not present, or removes it if it is — used to switch between light and dark themes.
<code>onclick="toggleTheme()"</code>	Event attribute that calls the <code>toggleTheme()</code> function whenever the button is clicked.
<code>document.getElementById("slideImg")</code>	Selects the image element by its id so its src can be changed from JavaScript.
<code>images[currentImage]</code>	Array of image URLs and an index variable used to keep track of which image is currently displayed.
<code>(currentImage + 1) % images.length</code>	Moves to the next image and wraps back to the first image after the last one using the modulus operator.
<code>document.createElement("li")</code>	Dynamically creates a new list item element in memory, which is then added to the page.
<code>appendChild()</code>	Adds a newly created element (like a task's <code></code>) as a child of an existing element (the <code></code>).
<code>input.value.trim()</code>	Reads the text typed into the input field and removes extra spaces before using it.
<code>removeBtn.onclick = function(){ li.remove(); }</code>	Assigns a click handler to the Remove button so that clicking it deletes its own list item from the page.
<code>oninput="countChars()"</code>	Event attribute that runs <code>countChars()</code> every time the user types or edits text in the textarea.
<code>textArea.value.length</code>	Gets the number of characters currently typed in the textarea.
<code>textContent</code>	Property used to update the visible text of an element, such as showing the live character count.

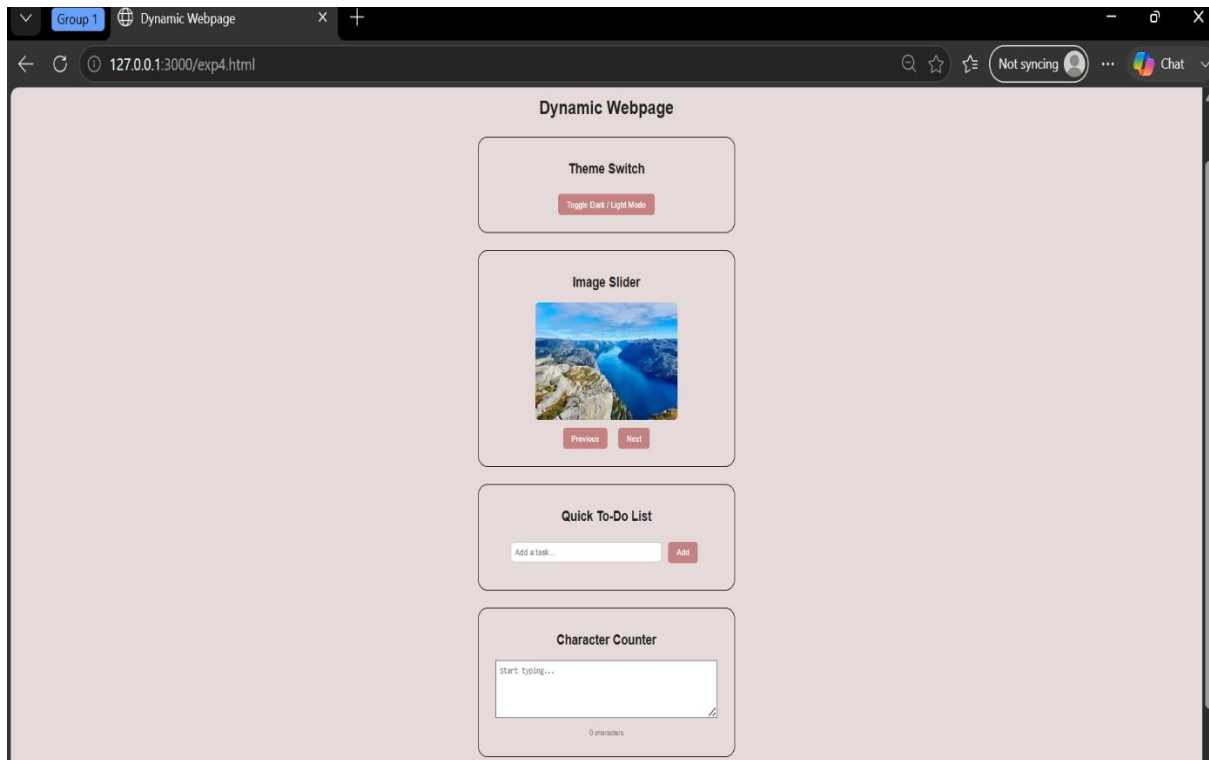
8. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Click "Toggle Dark / Light Mode" button	Page background and text colours switch between light and dark theme	Theme switched correctly on each click and reverted back on the next click	Pass
TC-02	Click "Next" and "Previous" buttons on the image slider	Image changes to the next/previous picture in the list, wrapping around at the ends	Image updated correctly in both directions, including wrap-around	Pass
TC-03	Type a task in the input box and click "Add"	Task appears as a new item in the to-do list with a Remove button	Task was added to the list correctly with a working Remove button	Pass
TC-04	Click "Remove" on a task in the to-do list	That task is deleted from the list	Task was removed from the list immediately	Pass

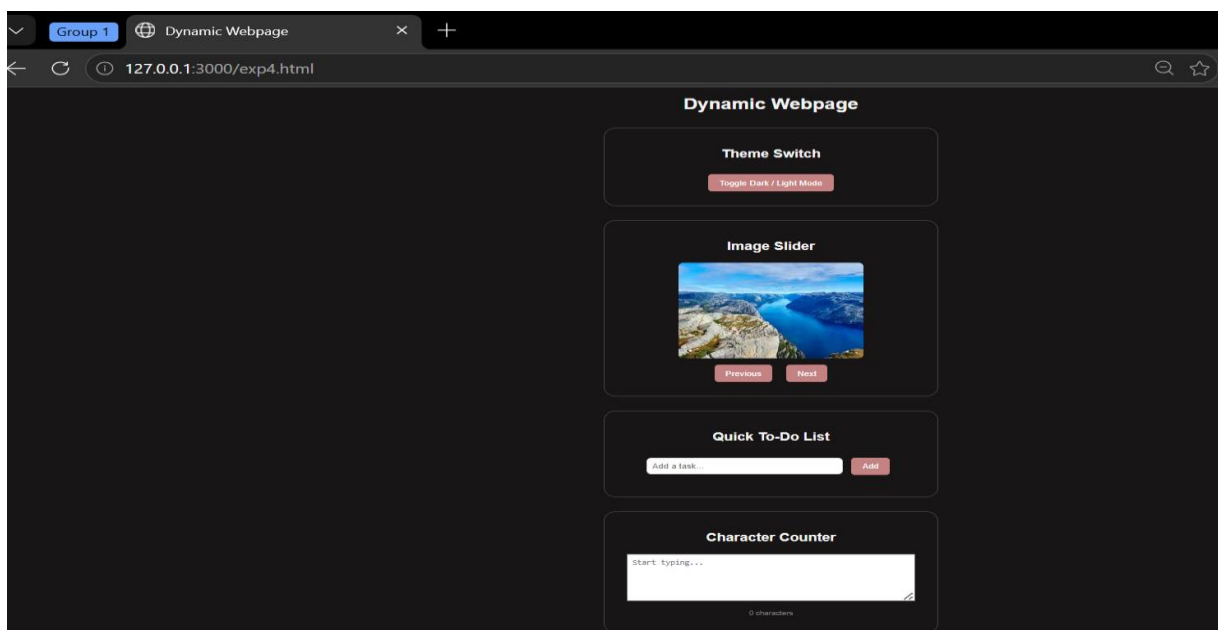
TC-05	Type text into the character counter textarea	Character count below the textarea updates live as text is typed	Character count updated correctly with every keystroke	Pass
-------	---	--	--	------

9. OUTPUT

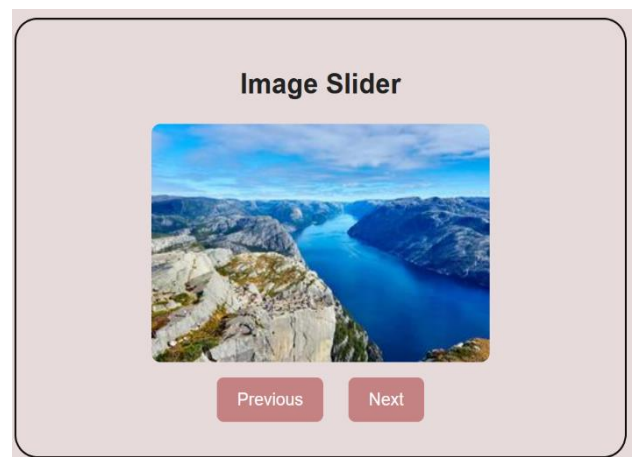
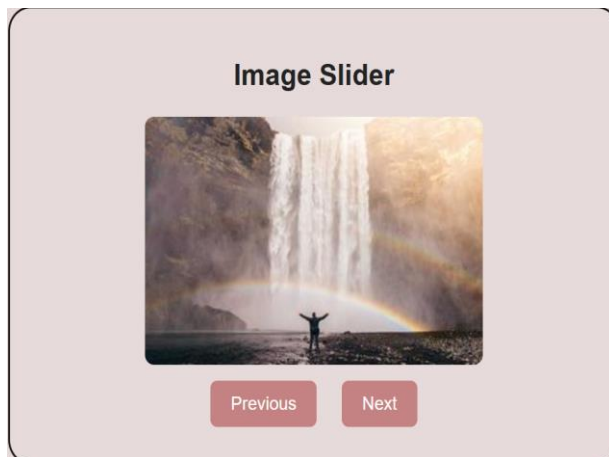
Output 1: Light Mode View



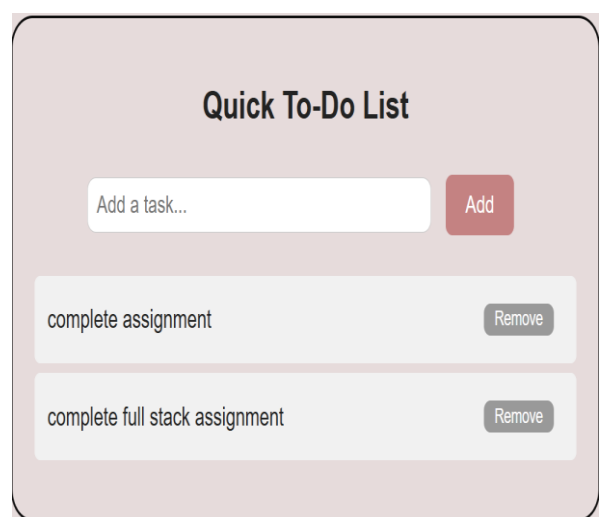
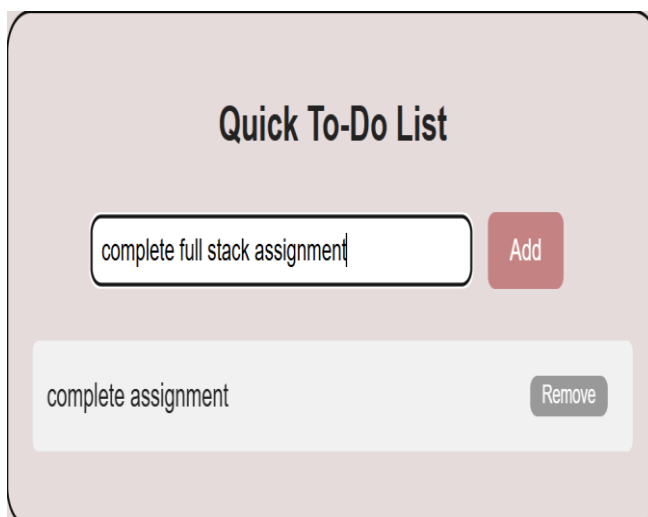
Output 2: Dark Mode View



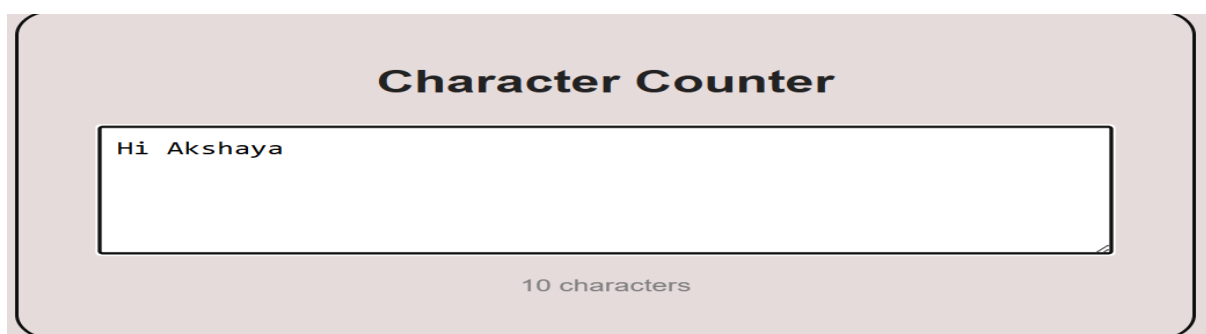
Output 3: Image Slider



Output 4: To-Do List with Tasks Added



Output 5: Character Counter in Use



10. RESULT

Thus, a dynamic webpage was successfully developed using JavaScript and the DOM. The theme toggle, image slider, to-do list, and live character counter were implemented and tested in a web browser, and all features were verified to work correctly.